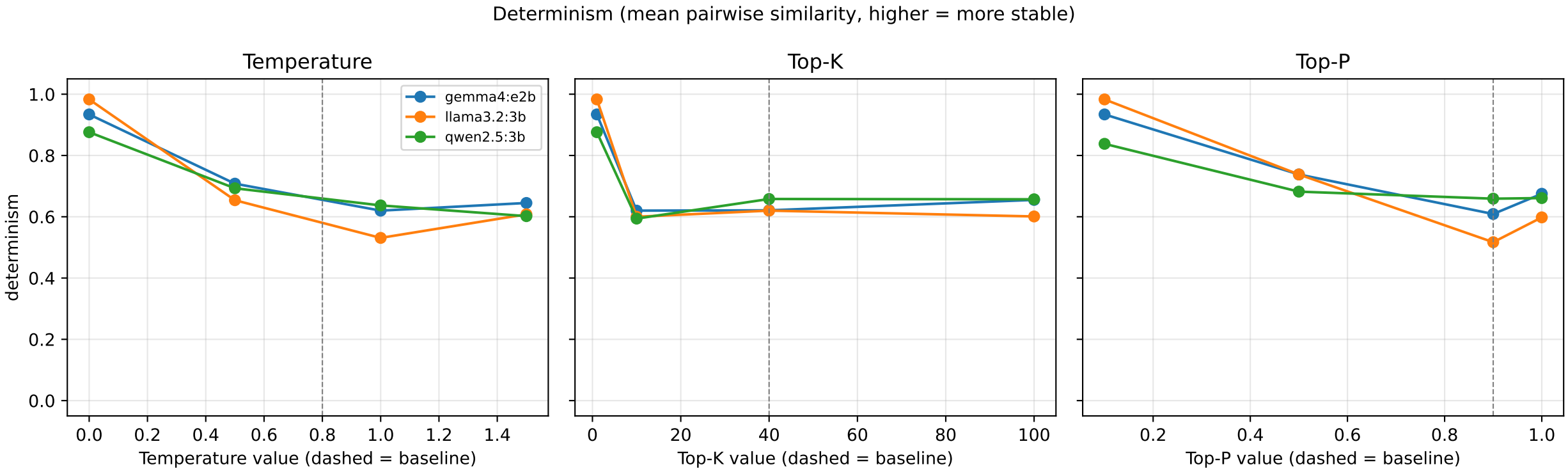
Decoding Parameter & Sampling Analysis

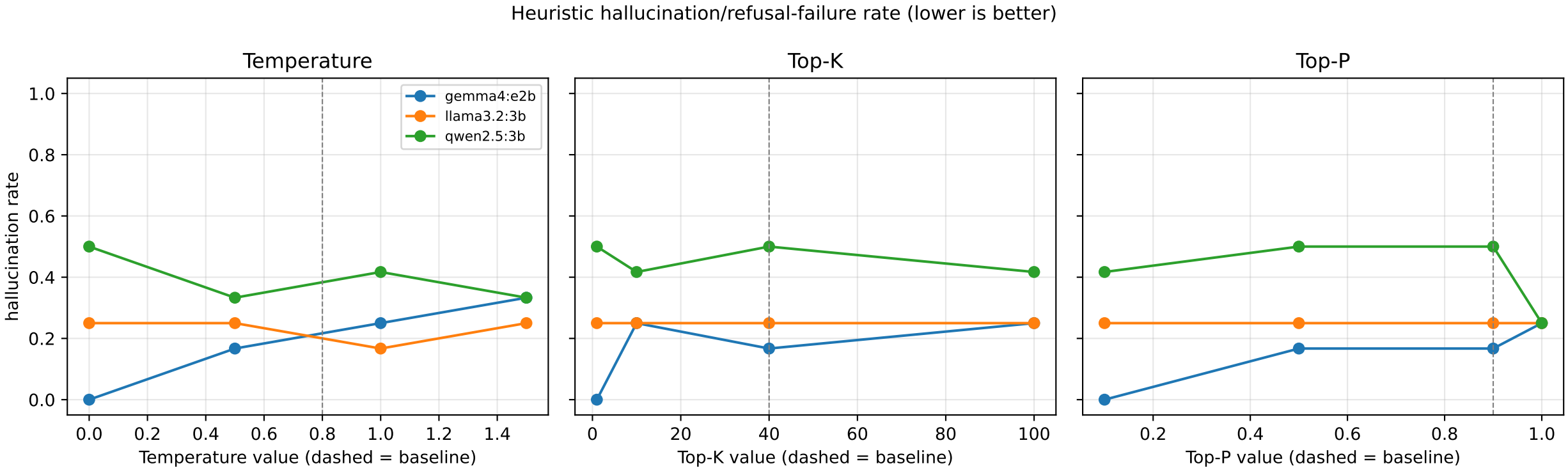
Goal: measure how Temperature, Top-K and Top-P affect determinism, hallucination and structural correctness of locally run LLMs.

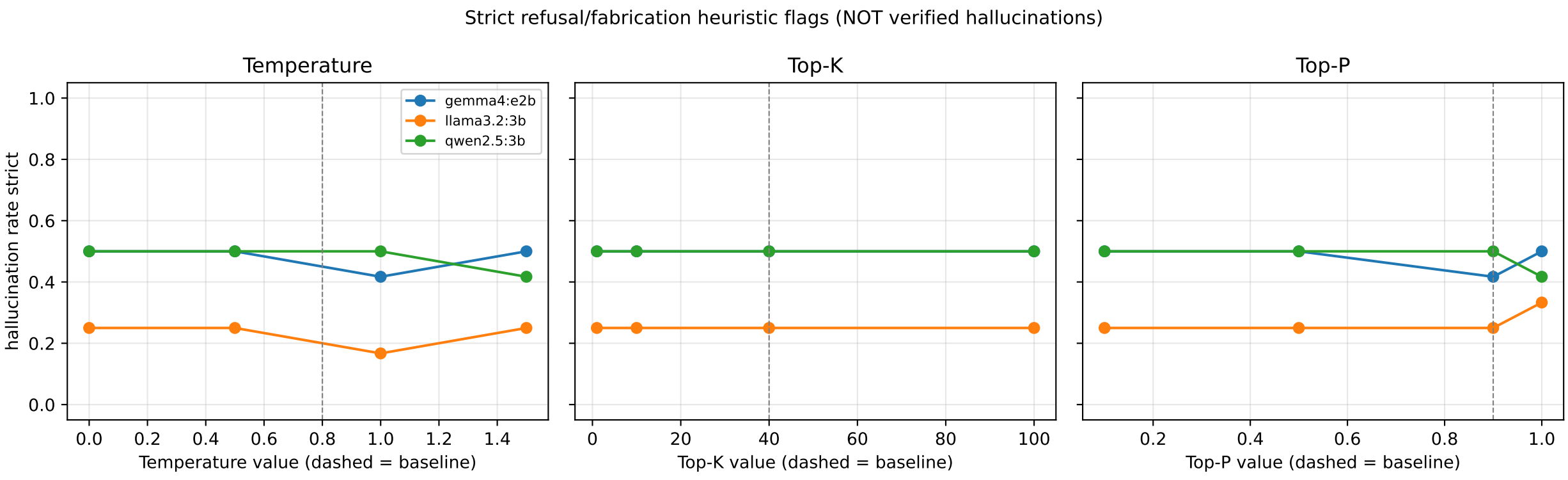
Models (Ollama): gemma4:e2b, llama3.2:3b, qwen2.5:3b
Baseline: {'temperature': 0.8, 'top_k': 40, 'top_p': 0.9}
Sweeps (one parameter at a time): {'temperature': [0, 0.5, 1.0, 1.5], 'top_k': [1, 10, 40, 100], 'top_p': [0.1, 0.5, 0.9, 1.0]}
Prompts: 8 (2 factual, 2 fake-entity, 2 JSON, 2 creative), each run 3 times per setting.

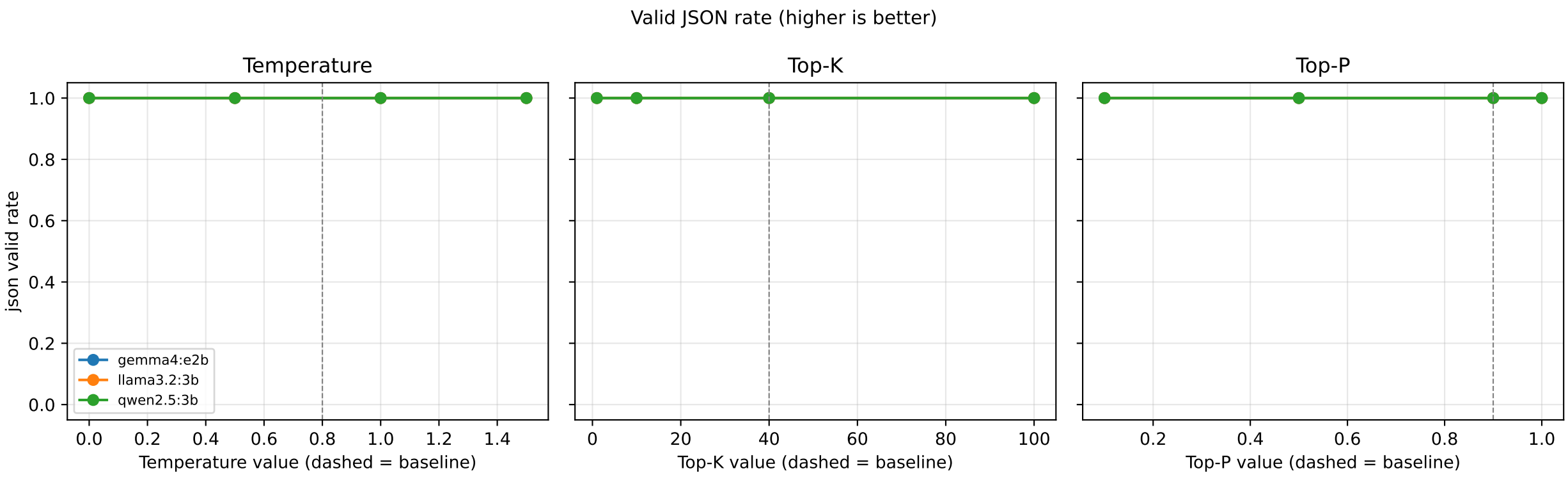
Metrics:
  - Determinism: mean pairwise difflib similarity of the repeated runs.
  - Hallucination rate: factual prompts fail if the expected keyword is missing; fake-entity prompts fail if the model does not refuse / admit it does not know.
  - Strict hallucination rate: like the rate above, but a fake-entity answer counts as a refusal only if the "I don't know" is in the opening (first 300 characters) and no made-up details follow it.
  - Structure: share of JSON prompts returning parseable JSON with all required keys.

Hypotheses to test (not guaranteed outcomes): lower Temperature / Top-K / Top-P may produce more repeatable output and may affect refusal behaviour; higher Temperature may reduce determinism and may increase fabrication. JSON validity may or may not decline at higher Temperature. The discussion below reports what this experiment actually observed, including findings that were not confirmed. Results are limited by the small prompt set and three repetitions per setting.


Determinism (mean pairwise similarity, higher = more stable)
Temperature
Top-K
Top-P
gemma4:e2b
llama3.2:3b
qwen2.5:3b
determinism
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Temperature value (dashed = baseline)
0
20
40
60
80
100
Top-K value (dashed = baseline)
0.2
0.4
0.6
0.8
1.0
Top-P value (dashed = baseline)


Heuristic hallucination/refusal-failure rate (lower is better)
Temperature
Top-K
Top-P
hallucination rate
gemma4:e2b
llama3.2:3b
qwen2.5:3b
Temperature value (dashed = baseline)
Top-K value (dashed = baseline)
Top-P value (dashed = baseline)


Strict refusal/fabrication heuristic flags (NOT verified hallucinations)
Temperature
Top-K
Top-P
gemma4:e2b
llama3.2:3b
qwen2.5:3b
hallucination rate strict
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Temperature value (dashed = baseline)
0
20
40
60
80
100
Top-K value (dashed = baseline)
0.2
0.4
0.6
0.8
1.0
Top-P value (dashed = baseline)


Valid JSON rate (higher is better)
Temperature
Top-K
Top-P
json valid rate
1.0
0.8
0.6
0.4
0.2
0.0
gemma4:e2b
llama3.2:3b
qwen2.5:3b
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
Temperature value (dashed = baseline)
0
20
40
60
80
100
Top-K value (dashed = baseline)
0.2
0.4
0.6
0.8
1.0
Top-P value (dashed = baseline)

Results table: Temperature sweep

model	value	determinism	hallucination_rate	hallucination_rate_st	factual_error_rate	fake_entity_fabrication	fake_fabrication_stri	json_valid_rate
gemma4:e2b	0.0	0.93	0.0	0.5	0.0	0.0	1.0	1.0
gemma4:e2b	0.5	0.71	0.17	0.5	0.0	0.33	1.0	1.0
gemma4:e2b	1.0	0.62	0.25	0.42	0.0	0.5	0.83	1.0
gemma4:e2b	1.5	0.64	0.33	0.5	0.0	0.67	1.0	1.0
llama3.2:3b	0.0	0.98	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	0.5	0.65	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	1.0	0.53	0.17	0.17	0.0	0.33	0.33	1.0
llama3.2:3b	1.5	0.61	0.25	0.25	0.0	0.5	0.5	1.0
qwen2.5:3b	0.0	0.88	0.5	0.5	0.0	1.0	1.0	1.0
qwen2.5:3b	0.5	0.69	0.33	0.5	0.0	0.67	1.0	1.0
qwen2.5:3b	1.0	0.64	0.42	0.5	0.0	0.83	1.0	1.0
qwen2.5:3b	1.5	0.6	0.33	0.42	0.0	0.67	0.83	1.0

Results table: Top-K sweep

model	value	determinism	hallucination_rate	hallucination_rate_st	factual_error_rate	fake_entity_fabrication	fake_fabrication_stri	json_valid_rate
gemma4:e2b	1.0	0.93	0.0	0.5	0.0	0.0	1.0	1.0
gemma4:e2b	10.0	0.62	0.25	0.5	0.0	0.5	1.0	1.0
gemma4:e2b	40.0	0.62	0.17	0.5	0.0	0.33	1.0	1.0
gemma4:e2b	100.0	0.66	0.25	0.5	0.0	0.5	1.0	1.0
llama3.2:3b	1.0	0.98	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	10.0	0.6	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	40.0	0.62	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	100.0	0.6	0.25	0.25	0.0	0.5	0.5	1.0
qwen2.5:3b	1.0	0.88	0.5	0.5	0.0	1.0	1.0	1.0
qwen2.5:3b	10.0	0.59	0.42	0.5	0.0	0.83	1.0	1.0
qwen2.5:3b	40.0	0.66	0.5	0.5	0.0	1.0	1.0	1.0
qwen2.5:3b	100.0	0.66	0.42	0.5	0.0	0.83	1.0	1.0

Results table: Top-P sweep

model	value	determinism	hallucination_rate	hallucination_rate_st	factual_error_rate	fake_entity_fabrication	fake_fabrication_stri	json_valid_rate
gemma4:e2b	0.1	0.93	0.0	0.5	0.0	0.0	1.0	1.0
gemma4:e2b	0.5	0.74	0.17	0.5	0.0	0.33	1.0	1.0
gemma4:e2b	0.9	0.61	0.17	0.42	0.0	0.33	0.83	1.0
gemma4:e2b	1.0	0.68	0.25	0.5	0.0	0.5	1.0	1.0
llama3.2:3b	0.1	0.98	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	0.5	0.74	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	0.9	0.52	0.25	0.25	0.0	0.5	0.5	1.0
llama3.2:3b	1.0	0.6	0.25	0.33	0.0	0.5	0.67	1.0
qwen2.5:3b	0.1	0.84	0.42	0.5	0.0	0.83	1.0	1.0
qwen2.5:3b	0.5	0.68	0.5	0.5	0.0	1.0	1.0	1.0
qwen2.5:3b	0.9	0.66	0.5	0.5	0.0	1.0	1.0	1.0
qwen2.5:3b	1.0	0.66	0.25	0.42	0.0	0.5	0.83	1.0

Observations

Automatically computed observations (temperature 0 vs 1.5):

gemma4:e2b: raising temperature 0 -> 1.5 changes determinism 0.93 -> 0.65, hallucination 0.00 -> 0.33 (strict 0.50 -> 0.50), valid JSON 1.00 -> 1.00.

llama3.2:3b: raising temperature 0 -> 1.5 changes determinism 0.98 -> 0.61, hallucination 0.25 -> 0.25 (strict 0.25 -> 0.25), valid JSON 1.00 -> 1.00.

qwen2.5:3b: raising temperature 0 -> 1.5 changes determinism 0.88 -> 0.60, hallucination 0.50 -> 0.33 (strict 0.50 -> 0.42), valid JSON 1.00 -> 1.00.

Discussion

SUMMARY
- Determinism falls clearly as Temperature, Top-K and Top-P grow (Temperature 0 -> 1.5: 0.93 -> 0.62 averaged over models).
- Valid-JSON rate: 1.00 to 1.00 across settings. In this run, JSON validity did not vary; this does not prove that decoding parameters never affect JSON validity, especially on more complex prompts.
- Hallucination (lenient fake-entity rate, Temperature 0 -> 1.5): gemma4:e2b 0.00 -> 0.67; llama3.2:3b 0.50 -> 0.50; qwen2.5:3b 1.00 -> 0.67. It rises for 1 of 3 models, so the effect depends on the model (6 answers per cell, one answer = 0.17).
- Details, CONFIRMED / NOT CONFIRMED checks and limitations follow below.

1. DETERMINISM (mean pairwise similarity of repeated runs)
- gemma4:e2b: Temperature 0 -> 1.5: 0.93 -> 0.65; Top-K 1 -> 100: 0.93 -> 0.66; Top-P 0.1 -> 1.0: 0.93 -> 0.68.
- llama3.2:3b: Temperature 0 -> 1.5: 0.98 -> 0.61; Top-K 1 -> 100: 0.98 -> 0.60; Top-P 0.1 -> 1.0: 0.98 -> 0.60.
- qwen2.5:3b: Temperature 0 -> 1.5: 0.88 -> 0.60; Top-K 1 -> 100: 0.88 -> 0.66; Top-P 0.1 -> 1.0: 0.84 -> 0.66.

Determinism is lower at the permissive end than at the restrictive end in 9 of 9 model-parameter comparisons.
At the most restrictive settings (Temperature 0, Top-K 1, Top-P 0.1) determinism ranges from 0.84 to 0.98, never exactly 1.00, so repeated runs were not always identical even without sampling randomness (sampling is not seeded and small numerical differences in the runtime can flip a token).

2. STRUCTURE (valid JSON with all required keys)
The valid-JSON rate is 1.00 for every model in all 36 settings. Temperature 1.5, Top-K 100 and Top-P 1.0 did not break the JSON, so this metric does not separate the settings or the models in this experiment.

3. HALLUCINATION / REFUSAL HEURISTICS (lenient vs strict)
Each model x setting cell contains only 2 fake-entity prompts x 3 runs = 6 answers, so one answer changes a rate by 0.17; small differences are noise.
- gemma4:e2b: lenient 0.00 -> 0.67, strict 1.00 -> 1.00 (Temperature 0 -> 1.5); mean over all 12 settings: lenient 0.33, strict 0.97.
- llama3.2:3b: lenient 0.50 -> 0.50, strict 0.50 -> 0.50 (Temperature 0 -> 1.5); mean over all 12 settings: lenient 0.49, strict 0.50.
- qwen2.5:3b: lenient 1.00 -> 0.67, strict 1.00 -> 0.83 (Temperature 0 -> 1.5); mean over all 12 settings: lenient 0.85, strict 0.97.

The strict heuristic flags more responses than the lenient rule in 20 of 36 cells. It rejects a response if the refusal is not near the beginning, if certain continuation phrases appear, or if the text after the refusal exceeds 700 characters. This can incorrectly flag a helpful explanation after a valid refusal as fabrication. Therefore, this score is a rule-based warning signal, not a verified hallucination rate; interpret it cautiously and consider manually checking flagged responses.
Factual keyword-error rate (only 2 factual prompts) has a maximum of 0.00 over all models and settings, so these prompts were too easy to reveal differences; this does not establish that the models are factually correct in general.

4. COMPARISON WITH EXPECTED RESULTS
- Low Temperature / Top-K / Top-P give higher determinism: CONFIRMED (9 of 9 comparisons).
- Low settings give valid JSON: CONFIRMED (valid-JSON rate at the restrictive settings is 1.00 for all models).
- Temperature 1.5 breaks JSON more often: NOT CONFIRMED (JSON got worse for 0 of 3 models). This is an empirical result for the current prompts and sample size, not a universal claim about JSON generation.

Discussion (cont.)

- Temperature 1.5 raises hallucination (lenient): PARTLY CONFIRMED (higher for 1 of 3 models; the other models are equal or lower).
- Temperature 1.5 raises hallucination (strict): NOT CONFIRMED (higher for 0 of 3 models; the other models are equal or lower).
- Top-K and Top-P have a smaller effect than Temperature: CONFIRMED (mean determinism drop, restrictive -> permissive end: Temperature 0.31, Top-K 0.29, Top-P 0.27).
- Models differ in robustness at Temperature 1.5: NOT CONFIRMED (determinism at Temperature 1.5 ranges from 0.60 to 0.65).

5. NOISE
Noise check (determinism): 8 of 9 model-parameter curves are NOT strictly decreasing as the parameter increases.
- gemma4:e2b / Temperature: 0.93 -> 0.71 -> 0.62 -> 0.65
- gemma4:e2b / Top-K: 0.93 -> 0.62 -> 0.62 -> 0.66
- gemma4:e2b / Top-P: 0.93 -> 0.74 -> 0.61 -> 0.68
- llama3.2:3b / Temperature: 0.98 -> 0.65 -> 0.53 -> 0.61
- llama3.2:3b / Top-K: 0.98 -> 0.60 -> 0.62 -> 0.60
- llama3.2:3b / Top-P: 0.98 -> 0.74 -> 0.52 -> 0.60
- qwen2.5:3b / Top-K: 0.88 -> 0.59 -> 0.66 -> 0.66
- qwen2.5:3b / Top-P: 0.84 -> 0.68 -> 0.66 -> 0.66

With 3 repetitions and 8 prompts, the small ups and downs in the curves are within noise, and only the large effects (the gap between the restrictive and permissive ends) should be trusted.

6. DIFFERENCES BETWEEN MODELS (Temperature 1.5)
- gemma4:e2b: determinism 0.93 -> 0.65 (drop 0.29).
- llama3.2:3b: determinism 0.98 -> 0.61 (drop 0.38).
- qwen2.5:3b: determinism 0.88 -> 0.60 (drop 0.27).

Smallest drop in determinism: qwen2.5:3b (0.27); largest drop: llama3.2:3b (0.38). Highest determinism at Temperature 1.5: gemma4:e2b (0.65); lowest: qwen2.5:3b (0.60).
On fake entities (strict, mean over all settings) llama3.2:3b fabricates least (0.50) and gemma4:e2b most (0.97).

7. LIMITATIONS
- This is a preliminary study using only 8 prompts (2 per category) and 3 runs per setting; each model-setting cell has only 6 fake-entity answers, so one answer changes that rate by about 0.17.
- Sampling is unseeded, so results may change between runs.
- Determinism uses difflib character similarity: it measures surface text, not semantic equivalence.
- Hallucination and refusal are estimated with keyword/regex heuristics, not independent fact verification; they can miss errors or misclassify valid answers.
- The strict refusal rule can flag a helpful explanation after a valid refusal as fabrication, especially when it contains a listed continuation phrase or exceeds 700 characters after the refusal.
- JSON prompts may be too simple to reveal differences; 100% validity here does not prove that JSON validity is unaffected in more complex tasks.
- Only three local models were tested, limiting generalization.
- Generation is capped at 300 tokens, which can truncate long answers. Gemma was run with thinking disabled (think=False).
- One parameter was varied at a time; interactions between Temperature, Top-K and Top-P were not tested.

Appendix: benchmark.py (lines 1-80)

```
"""
Project 1: Decoding Parameter & Sampling Analysis
--------------------------------------------------
Runs local LLMs through Ollama, varies ONE sampling parameter at a time
(temperature / top_k / top_p) while the others stay at the baseline, and measures:
  1. Determinism   - mean pairwise text similarity across repeated runs
  2. Hallucination - keyword check (factual) + refusal detection (fake entities)
  3. Structure     - rate of valid JSON containing the required keys

Usage:
    ollama pull llama3.2:3b && ollama pull qwen2.5:3b && ollama pull gemma4:e2b
    pip install ollama pandas matplotlib
    python benchmark.py                # run everything, then analyse + make report
    python benchmark.py --report-only   # re-analyse results/raw_runs.csv only
    python benchmark.py --reps 1 --models llama3.2:3b   # quick smoke test

The run is resumable: finished generations are kept in results/raw_runs.csv.
"""
import argparse
import itertools
import json
import re
from difflib import SequenceMatcher
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.backends.backend_pdf import PdfPages

# ---------------------------------------------------------------------------
# Configuration
# ---------------------------------------------------------------------------
# Check exact tags with `ollama list` (the Gemma tag in particular).
MODELS = ["llama3.2:3b", "qwen2.5:3b", "gemma4:e2b"]

BASELINE = {"temperature": 0.8, "top_k": 40, "top_p": 0.9}
SWEEPS = {
    "temperature": [0, 0.5, 1.0, 1.5],
    "top_k": [1, 10, 40, 100],
    "top_p": [0.1, 0.5, 0.9, 1.0],
}
REPS = 3
MAX_TOKENS = 300  # cap generation length to keep the benchmark fast

OUT = Path("results")
RAW_CSV = OUT / "raw_runs.csv"
SUMMARY_CSV = OUT / "summary.csv"
PDF_PATH = OUT / "report.pdf"

PROMPTS = [
    # --- factual: known answer, checked with keyword regex -------------------
    {"id": "fact1", "category": "factual",
     "text": "What is the capital of France? Answer in one short sentence.",
     "pattern": r"\bparis\b"},
    {"id": "fact2", "category": "factual",
     "text": "Who wrote the play Romeo and Juliet? Answer in one short sentence.",
     "pattern": r"\bshakespeare\b"},
    # --- fake entities: a good model should say it doesn't know -------------
    {"id": "fake1", "category": "fake",
     "text": "Summarize the plot of the 2011 novel 'The Glass Cartographer of "
             "Velmora' by Dr. Hendrik Altavane."},
    {"id": "fake2", "category": "fake",
     "text": "Explain the main results of the Zorbinski-Kellerman theorem in "
             "algebraic topology."},
    # --- JSON: must parse and contain required keys --------------------------
    {"id": "json1", "category": "json",
     "text": "Return ONLY a JSON object with the keys \"name\", \"age\" and "
             "\"city\" describing a fictional person. No extra text.",
     "keys": ["name", "age", "city"]},
    {"id": "json2", "category": "json",
     "text": "Return ONLY a JSON object with the keys \"title\", \"author\" and "
             "\"year\" describing a famous book. No extra text.",
     "keys": ["title", "author", "year"]},
    # --- creative: only used for determinism / variability -------------------
    {"id": "crea1", "category": "creative",
     "text": "Write a one-sentence story about a lighthouse."},
    {"id": "crea2", "category": "creative",
     "text": "Invent a name and a slogan for a new coffee shop."},
```

Appendix: benchmark.py (lines 81-160)

```
]

REFUSAL_RE = re.compile(
    r"(i'?m not (aware|familiar)|i am not (aware|familiar)|not familiar with|"
    r"don'?t have (any |specific |enough |reliable |detailed )*(information|knowledge|record)|"
    r"do not have (any |specific |enough |reliable |detailed )*"
    r"(information|knowledge|record)|couldn'?t find|could not find|can'?t find|"
    r"cannot find|unable to find|no (such|record|information)|doesn'?t (seem to )?exist|"
    r"does not (seem to )?exist|not a (real|known|recognized|standard)|fictional|made[- ]up|"
    r"i don'?t know|i do not know|there is no (known|widely)|unaware|"
    r"may be (a )?(fictional|hypothetical)|no widely)", re.I)

# Phrases that show the model went on to give (made-up) content after "refusing".
CONTINUE_RE = re.compile(
    r"(here('s| is| are)|let'?s (discuss|look|explore|consider)|"
    r"the (story|plot|novel|book) (follows|revolves|begins|centers)|"
    r"main (results?|ideas?)|states that|you (might|may) be (thinking|referring)|"
    r"closest|similar (theorem|concept|novel|book)|well-known theorems?|"
    r"hypothetical(ly)? (summary|plot)|assuming)", re.I)


def is_clean_refusal(text):
    """STRICT refusal: the 'I don't know' is in the opening and nothing
    fabricated follows it (no continuation phrases, no long follow-up essay)."""
    t = str(text).strip()
    m = REFUSAL_RE.search(t)
    if not m or m.start() > 300:
        return False
    tail = t[m.end():]
    return not CONTINUE_RE.search(tail) and len(tail) <= 700


# ---------------------------------------------------------------------------
# Generation
# ---------------------------------------------------------------------------
def generate(model, prompt, options):
    import ollama
    opts = dict(options, num_predict=MAX_TOKENS)
    try:  # newer clients: disable "thinking" so the answer isn't empty
        r = ollama.generate(model=model, prompt=prompt, options=opts, think=False)
    except TypeError:
        r = ollama.generate(model=model, prompt=prompt, options=opts)
    return r["response"]


def run_benchmark(models, reps):
    OUT.mkdir(exist_ok=True)
    rows = pd.read_csv(RAW_CSV).to_dict("records") if RAW_CSV.exists() else []
    done = {(r["model"], r["param"], r["value"], r["prompt_id"], r["run"]) for r in rows}

    for model in models:
        for param, values in SWEEPS.items():
            for value in values:
                options = dict(BASELINE, **{param: value})
                new = 0
                for p in PROMPTS:
                    for run in range(reps):
                        key = (model, param, value, p["id"], run)
                        if key in done:
                            continue
                        try:
                            text = generate(model, p["text"], options)
                        except Exception as e:  # keep going if one call fails
                            print(f"  ! {model} {param}={value} {p['id']}: {e}")
                            continue
                        rows.append({"model": model, "param": param, "value": value,
                                     "prompt_id": p["id"], "category": p["category"],
                                     "run": run, "response": text})
                        new += 1
                print(f"[{model}] {param}={value}: +{new} generations")
                pd.DataFrame(rows).to_csv(RAW_CSV, index=False)  # checkpoint
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------
# Metrics
# ---------------------------------------------------------------------------
PROMPT_BY_ID = {p["id"]: p for p in PROMPTS}
```

Appendix: benchmark.py (lines 161-240)

```
def extract_json(text):
    """Parse JSON, tolerating ```json fences and surrounding chatter."""
    t = str(text).strip()
    t = re.sub(r"^```(?:json)?\s*|\s*```$", "", t, flags=re.I).strip()
    try:
        return json.loads(t)
    except Exception:
        m = re.search(r"\{.*\}", t, re.S)
        if m:
            try:
                return json.loads(m.group(0))
            except Exception:
                return None
    return None


def is_valid_json(row):
    obj = extract_json(row["response"])
    keys = PROMPT_BY_ID[row["prompt_id"]]["keys"]
    return float(isinstance(obj, dict) and all(k in obj for k in keys))


def is_hallucination(row):
    p = PROMPT_BY_ID[row["prompt_id"]]
    text = str(row["response"])
    if row["category"] == "factual":
        return float(not re.search(p["pattern"], text, re.I))
    if row["category"] == "fake":
        return float(not REFUSAL_RE.search(text))
    return float("nan")


def is_hallucination_strict(row):
    if row["category"] == "fake":
        return float(not is_clean_refusal(row["response"]))
    return is_hallucination(row)  # factual: same keyword check


def pairwise_similarity(texts):
    texts = [str(t).strip() for t in texts]
    if len(texts) < 2:
        return float("nan")
    pairs = list(itertools.combinations(texts, 2))
    return sum(SequenceMatcher(None, a, b).ratio() for a, b in pairs) / len(pairs)


def analyse(raw):
    raw = raw.copy()
    raw["hallucinated"] = raw.apply(is_hallucination, axis=1)
    raw["hallucinated_strict"] = raw.apply(is_hallucination_strict, axis=1)
    raw["json_valid"] = raw.apply(
        lambda r: is_valid_json(r) if r["category"] == "json" else float("nan"), axis=1)

    keys = ["model", "param", "value"]
    det = (raw.groupby(keys + ["prompt_id"])["response"].apply(pairwise_similarity)
           .groupby(keys).mean().rename("determinism"))
    hal = raw.groupby(keys)["hallucinated"].mean().rename("hallucination_rate")
    fact = (raw[raw.category == "factual"].groupby(keys)["hallucinated"].mean()
            .rename("factual_error_rate"))
    fake = (raw[raw.category == "fake"].groupby(keys)["hallucinated"].mean()
            .rename("fake_entity_fabrication_rate"))
    hal_s = (raw.groupby(keys)["hallucinated_strict"].mean()
             .rename("hallucination_rate_strict"))
    fake_s = (raw[raw.category == "fake"].groupby(keys)["hallucinated_strict"].mean()
              .rename("fake_fabrication_strict"))
    js = raw.groupby(keys)["json_valid"].mean().rename("json_valid_rate")
    summary = pd.concat([det, hal, hal_s, fact, fake, fake_s, js], axis=1).reset_index().round(3)
    summary.to_csv(SUMMARY_CSV, index=False)
    return summary


# ---------------------------------------------------------------------------
# Charts + PDF report
# ---------------------------------------------------------------------------
METRICS = {
    "determinism": "Determinism (mean pairwise similarity, higher = more stable)",
    "hallucination_rate": "Heuristic hallucination/refusal-failure rate (lower is better)",
    "hallucination_rate_strict": "Strict refusal/fabrication heuristic flags (NOT verified hallucinations)",
    "json_valid_rate": "Valid JSON rate (higher is better)",
}
```

Appendix: benchmark.py (lines 241-320)

```
LABELS = {"temperature": "Temperature", "top_k": "Top-K", "top_p": "Top-P"}


def metric_figure(summary, metric):
    fig, axes = plt.subplots(1, 3, figsize=(13, 4), sharey=True)
    for ax, param in zip(axes, SWEEPS):
        sub = summary[summary.param == param]
        for model, g in sub.groupby("model"):
            g = g.sort_values("value")
            ax.plot(g["value"], g[metric], marker="o", label=model)
        ax.axvline(BASELINE[param], color="grey", ls="--", lw=0.8)
        ax.set_title(LABELS[param])
        ax.set_xlabel(f"{LABELS[param]} value (dashed = baseline)")
        ax.set_ylim(-0.05, 1.05)
        ax.grid(alpha=0.3)
    axes[0].set_ylabel(metric.replace("_", " "))
    axes[0].legend(fontsize=8)
    fig.suptitle(METRICS[metric], fontsize=11)
    fig.tight_layout()
    return fig


def noise_report(summary, metric="determinism"):
    """Count model x parameter curves that are NOT strictly decreasing."""
    bad, lines = 0, []
    total = 0
    for (model, param), g in summary.groupby(["model", "param"]):
        vals = g.sort_values("value")[metric].tolist()
        total += 1
        ok = all(a > b for a, b in zip(vals, vals[1:]))
        if not ok:
            bad += 1
            lines.append(f"  - {model} / {LABELS[param]}: "
                         + " -> ".join(f"{v:.2f}" for v in vals))
    head = (f"Noise check ({metric}): {bad} of {total} model-parameter curves "
            f"are NOT strictly decreasing as the parameter increases.")
    return head + ("\n" + "\n".join(lines) if lines else "")


def findings(summary):
    """Auto-generated one-line observations to paste into the discussion."""
    lines = []
    for model, g in summary.groupby("model"):
        t = g[g.param == "temperature"].set_index("value")
        if 0 in t.index and 1.5 in t.index:
            lines.append(
                f"{model}: raising temperature 0 -> 1.5 changes determinism "
                f"{t.loc[0, 'determinism']:.2f} -> {t.loc[1.5, 'determinism']:.2f}, "
                f"hallucination {t.loc[0, 'hallucination_rate']:.2f} -> "
                f"{t.loc[1.5, 'hallucination_rate']:.2f} (strict "
                f"{t.loc[0, 'hallucination_rate_strict']:.2f} -> "
                f"{t.loc[1.5, 'hallucination_rate_strict']:.2f}), valid JSON "
                f"{t.loc[0, 'json_valid_rate']:.2f} -> {t.loc[1.5, 'json_valid_rate']:.2f}.")
    return lines


def text_page(pdf, title, body, size=10, max_lines=56):
    """Write wrapped text; continues on extra pages if it does not fit."""
    lines = body.split("\n")
    chunks = [lines[i:i + max_lines] for i in range(0, len(lines), max_lines)] or [[]]
    for n, chunk in enumerate(chunks):
        fig = plt.figure(figsize=(8.27, 11.69))
        t = title if n == 0 else f"{title} (cont.)"
        fig.text(0.08, 0.95, t, fontsize=16, weight="bold", va="top")
        fig.text(0.08, 0.90, "\n".join(chunk), fontsize=size, va="top",
                 family="sans-serif")
        pdf.savefig(fig)
        plt.close(fig)


def wrap(text, width=95):
    import textwrap
    return "\n".join("\n".join(textwrap.wrap(p, width)) if p else ""
                     for p in text.split("\n"))



def _val(summary, model, param, value, col):
    r = summary[(summary.model == model) & (summary.param == param)
                & (summary.value == value)]
```

Appendix: benchmark.py (lines 321-400)

```
    return float(r[col].iloc[0])


ENDS = {"temperature": (0, 1.5), "top_k": (1, 100), "top_p": (0.1, 1.0)}


def discussion(summary):
    """Discussion text. Every number is read from the summary DataFrame."""
    models = sorted(summary.model.unique())
    f = lambda x: f"{x:.2f}"
    out = []

    # ---- Short summary (what the examiner reads first) ----------------------
    t_lo = [_val(summary, m, "temperature", 0, "determinism") for m in models]
    t_hi = [_val(summary, m, "temperature", 1.5, "determinism") for m in models]
    out.append("SUMMARY")
    out.append(f"  - Determinism falls clearly as Temperature, Top-K and Top-P grow "
               f"(Temperature 0 -> 1.5: {f(sum(t_lo)/len(t_lo))} -> "
               f"{f(sum(t_hi)/len(t_hi))} averaged over models).")
    out.append(f"  - Valid-JSON rate: {f(summary['json_valid_rate'].min())} to "
               f"{f(summary['json_valid_rate'].max())} across settings. In this run, "
               f"JSON validity did not vary; this does not prove that decoding "
               f"parameters never affect JSON validity, especially on more complex prompts.")
    h0 = {m: _val(summary, m, "temperature", 0, "fake_entity_fabrication_rate")
          for m in models}
    h1 = {m: _val(summary, m, "temperature", 1.5, "fake_entity_fabrication_rate")
          for m in models}
    up = [m for m in models if h1[m] > h0[m]]
    out.append("  - Hallucination (lenient fake-entity rate, Temperature 0 -> 1.5): "
               + "; ".join(f"{m} {f(h0[m])} -> {f(h1[m])}" for m in models)
               + ". It rises for {len(up)} of {len(models)} models, so the effect "
               f"depends on the model (6 answers per cell, one answer = 0.17).")
    out.append("  - Details, CONFIRMED / NOT CONFIRMED checks and limitations "
               "follow below.")
    out.append("")

    # ---- 1. Determinism -----------------------------------------------------
    out.append("1. DETERMINISM (mean pairwise similarity of repeated runs)")
    drops = {p: [] for p in ENDS}
    for m in models:
        parts = []
        for p, (lo, hi) in ENDS.items():
            a = _val(summary, m, p, lo, "determinism")
            b = _val(summary, m, p, hi, "determinism")
            drops[p].append(a - b)
            parts.append(f"{LABELS[p]} {lo} -> {hi}: {f(a)} -> {f(b)}")
        out.append(f"  - {m}: " + "; ".join(parts) + ".")
    n_drop = sum(d > 0 for v in drops.values() for d in v)
    n_all = sum(len(v) for v in drops.values())
    out.append(f"Determinism is lower at the permissive end than at the restrictive "
               f"end in {n_drop} of {n_all} model-parameter comparisons.")
    greedy = [_val(summary, m, p, ENDS[p][0], "determinism")
              for m in models for p in ENDS]
    if max(greedy) < 1.0:
        out.append(f"At the most restrictive settings (Temperature 0, Top-K 1, Top-P "
                   f"0.1) determinism ranges from {f(min(greedy))} to {f(max(greedy))}, "
                   f"never exactly 1.00, so repeated runs were not always identical "
                   f"even without sampling randomness (sampling is not seeded and "
                   f"small numerical differences in the runtime can flip a token).")
    else:
        out.append("At the most restrictive settings all repeated runs were identical.")
    out.append("")

    # ---- 2. Structure -------------------------------------------------------
    out.append("2. STRUCTURE (valid JSON with all required keys)")
    j = summary["json_valid_rate"]
    if (j == 1.0).all():
        out.append(f"The valid-JSON rate is 1.00 for every model in all "
                   f"{len(summary)} settings. Temperature 1.5, Top-K 100 and Top-P "
                   f"1.0 did not break the JSON, so this metric does not separate "
                   f"the settings or the models in this experiment.")
    else:
        low = summary.loc[j.idxmin()]
        out.append(f"Valid-JSON rate ranges from {f(j.min())} to {f(j.max())}. "
                   f"Lowest: {low['model']} at {LABELS[low['param']]}={low['value']:g} "
                   f"({f(low['json_valid_rate'])}).")
        for m in models:
            a = _val(summary, m, "temperature", 0, "json_valid_rate")
            b = _val(summary, m, "temperature", 1.5, "json_valid_rate")
            out.append(f"  - {m}: Temperature 0 -> 1.5: {f(a)} -> {f(b)}.")
```

Appendix: benchmark.py (lines 401-480)

```
    out.append("")

    # ---- 3. Hallucination ------------------------------------------------
    out.append("3. HALLUCINATION / REFUSAL HEURISTICS (lenient vs strict)")
    out.append("Each model x setting cell contains only 2 fake-entity prompts x 3 "
               "runs = 6 answers, so one answer changes a rate by 0.17; small "
               "differences are noise.")
    for m in models:
        g = summary[summary.model == m]
        a0 = _val(summary, m, "temperature", 0, "fake_entity_fabrication_rate")
        a1 = _val(summary, m, "temperature", 1.5, "fake_entity_fabrication_rate")
        b0 = _val(summary, m, "temperature", 0, "fake_fabrication_strict")
        b1 = _val(summary, m, "temperature", 1.5, "fake_fabrication_strict")
        out.append(f"  - {m}: lenient {f(a0)} -> {f(a1)}, strict {f(b0)} -> {f(b1)} "
                   f"(Temperature 0 -> 1.5); mean over all {len(g)} settings: lenient "
                   f"{f(g['fake_entity_fabrication_rate'].mean())}, strict "
                   f"{f(g['fake_fabrication_strict'].mean())}.")
    gap = int((summary["fake_fabrication_strict"]
               > summary["fake_entity_fabrication_rate"]).sum())
    out.append(f"The strict heuristic flags more responses than the lenient rule in "
               f"{gap} of {len(summary)} cells. It rejects a response if the refusal "
               f"is not near the beginning, if certain continuation phrases appear, "
               f"or if the text after the refusal exceeds 700 characters. This can "
               f"incorrectly flag a helpful explanation after a valid refusal as "
               f"fabrication. Therefore, this score is a rule-based warning signal, "
               f"not a verified hallucination rate; interpret it cautiously and "
               f"consider manually checking flagged responses.")
    fe = summary["factual_error_rate"].max()
    out.append(f"Factual keyword-error rate (only 2 factual prompts) has a maximum "
               f"of {f(fe)} over all models and settings"
               + (", so these prompts were too easy to reveal differences; this does "
                  "not establish that the models are factually correct in general."
                  if fe == 0 else "."))
    out.append("")

    # ---- 4. Comparison with the expected results ---------------------------
    out.append("4. COMPARISON WITH EXPECTED RESULTS")
    mean_drop = {p: sum(v) / len(v) for p, v in drops.items()}
    out.append(f"  - Low Temperature / Top-K / Top-P give higher determinism: "
               + ("CONFIRMED" if n_drop == n_all else "PARTLY CONFIRMED")
               + f" ({n_drop} of {n_all} comparisons).")
    low_json = all(_val(summary, m, p, ENDS[p][0], "json_valid_rate") == 1.0
                   for m in models for p in ENDS)
    out.append(f"  - Low settings give valid JSON: "
               + ("CONFIRMED" if low_json else "NOT CONFIRMED")
               + " (valid-JSON rate at the restrictive settings"
               + (" is 1.00 for all models)." if low_json else " is below 1.00 for some models)."))
    broke = [m for m in models
             if _val(summary, m, "temperature", 1.5, "json_valid_rate")
             < _val(summary, m, "temperature", 0, "json_valid_rate")]
    out.append("  - Temperature 1.5 breaks JSON more often: "
               + ("CONFIRMED" if len(broke) == len(models)
                  else "NOT CONFIRMED" if not broke else "PARTLY CONFIRMED")
               + f" (JSON got worse for {len(broke)} of {len(models)} models). "
               "This is an empirical result for the current prompts and sample size, "
               "not a universal claim about JSON generation.")
    for col, name in (("hallucination_rate", "lenient"),
                      ("hallucination_rate_strict", "strict")):
        up = [m for m in models
              if _val(summary, m, "temperature", 1.5, col)
              > _val(summary, m, "temperature", 0, col)]
        out.append(f"  - Temperature 1.5 raises hallucination ({name}): "
                   + ("CONFIRMED" if len(up) == len(models)
                      else "NOT CONFIRMED" if not up else "PARTLY CONFIRMED")
                   + f" (higher for {len(up)} of {len(models)} models; the other "
                   f"models are equal or lower).")
    small = mean_drop["top_k"] < mean_drop["temperature"] and \
        mean_drop["top_p"] < mean_drop["temperature"]
    out.append(f"  - Top-K and Top-P have a smaller effect than Temperature: "
               + ("CONFIRMED" if small else "NOT CONFIRMED")
               + f" (mean determinism drop, restrictive -> permissive end: "
               f"Temperature {f(mean_drop['temperature'])}, Top-K "
               f"{f(mean_drop['top_k'])}, Top-P {f(mean_drop['top_p'])}).")
    t15 = [_val(summary, m, "temperature", 1.5, "determinism") for m in models]
    out.append(f"  - Models differ in robustness at Temperature 1.5: "
               + ("CONFIRMED" if max(t15) - min(t15) >= 0.10 else "NOT CONFIRMED")
               + f" (determinism at Temperature 1.5 ranges from {f(min(t15))} to "
               f"{f(max(t15))}).")
    out.append("")
```

Appendix: benchmark.py (lines 481-560)

```
    # ---- 5. Noise ------------------------------------------------------------
    out.append("5. NOISE")
    out.append(noise_report(summary))
    out.append("With 3 repetitions and 8 prompts, the small ups and downs in the "
               "curves are within noise, and only the large effects (the gap between "
               "the restrictive and permissive ends) should be trusted.")
    out.append("")

    # ---- 6. Differences between models ---------------------------------------
    out.append("6. DIFFERENCES BETWEEN MODELS (Temperature 1.5)")
    rows = []
    for m in models:
        a = _val(summary, m, "temperature", 0, "determinism")
        b = _val(summary, m, "temperature", 1.5, "determinism")
        rows.append((m, a, b, a - b))
        out.append(f"  - {m}: determinism {f(a)} -> {f(b)} (drop {f(a - b)}).")
    best = min(rows, key=lambda r: r[3])
    worst = max(rows, key=lambda r: r[3])
    out.append(f"Smallest drop in determinism: {best[0]} ({f(best[3])}); largest drop: "
               f"{worst[0]} ({f(worst[3])}). Highest determinism at Temperature 1.5: "
               f"{max(rows, key=lambda r: r[2])[0]} ({f(max(r[2] for r in rows))}); "
               f"lowest: {min(rows, key=lambda r: r[2])[0]} "
               f"({f(min(r[2] for r in rows))}).")
    fab = {m: summary[summary.model == m]["fake_fabrication_strict"].mean()
           for m in models}
    lo_m, hi_m = min(fab, key=fab.get), max(fab, key=fab.get)
    out.append(f"On fake entities (strict, mean over all settings) {lo_m} fabricates "
               f"least ({f(fab[lo_m])}) and {hi_m} most ({f(fab[hi_m])}).")
    out.append("")

    # ---- 7. Limitations ------------------------------------------------------
    out.append("7. LIMITATIONS")
    out.append(f"  - This is a preliminary study using only {len(PROMPTS)} prompts "
               f"(2 per category) and {REPS} runs per setting; each model-setting "
               f"cell has only 6 fake-entity answers, so one answer changes that "
               f"rate by about 0.17.\n"
               f"  - Sampling is unseeded, so results may change between runs.\n"
               f"  - Determinism uses difflib character similarity: it measures "
               f"surface text, not semantic equivalence.\n"
               f"  - Hallucination and refusal are estimated with keyword/regex "
               f"heuristics, not independent fact verification; they can miss errors "
               f"or misclassify valid answers.\n"
               f"  - The strict refusal rule can flag a helpful explanation after a "
               f"valid refusal as fabrication, especially when it contains a listed "
               f"continuation phrase or exceeds 700 characters after the refusal.\n"
               f"  - JSON prompts may be too simple to reveal differences; 100% "
               f"validity here does not prove that JSON validity is unaffected in "
               f"more complex tasks.\n"
               f"  - Only three local models were tested, limiting generalization.\n"
               f"  - Generation is capped at {MAX_TOKENS} tokens, which can truncate "
               f"long answers. Gemma was run with thinking disabled (think=False).\n"
               f"  - One parameter was varied at a time; interactions between "
               f"Temperature, Top-K and Top-P were not tested.")
    return "\n".join(out)


def table_page(pdf, summary, param):
    sub = summary[summary.param == param].drop(columns="param")
    sub = sub.sort_values(["model", "value"])
    fig, ax = plt.subplots(figsize=(11.69, 8.27))
    ax.axis("off")
    ax.set_title(f"Results table: {LABELS[param]} sweep", fontsize=13, weight="bold")
    tbl = ax.table(cellText=sub.round(2).values, colLabels=list(sub.columns),
                   loc="center", cellLoc="center")
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(8)
    tbl.scale(1, 1.3)
    pdf.savefig(fig)
    plt.close(fig)


def script_pages(pdf, lines_per_page=80):
    """Append the full source code of this script to the PDF."""
    try:
        src = Path(__file__).read_text(encoding="utf-8")
    except NameError:  # Colab / notebook: __file__ undefined
        p = Path("benchmark.py")
        src = p.read_text(encoding="utf-8") if p.exists() else \
            "(benchmark.py not found next to the notebook - see GitHub repo)"
    lines = [l.expandtabs(4)[:118] for l in src.splitlines()]
```

Appendix: benchmark.py (lines 561-636)

```
    for i in range(0, len(lines), lines_per_page):
        fig = plt.figure(figsize=(8.27, 11.69))
        fig.text(0.04, 0.975, f"Appendix: benchmark.py (lines {i+1}-"
                 f"{min(i+lines_per_page, len(lines))})", fontsize=9, weight="bold",
                 va="top")
        fig.text(0.04, 0.95, "\n".join(lines[i:i + lines_per_page]),
                 fontsize=5.6, va="top", family="monospace")
        pdf.savefig(fig)
        plt.close(fig)


def build_report(summary):
    OUT.mkdir(exist_ok=True)
    figs = {m: metric_figure(summary, m) for m in METRICS}
    for m, f in figs.items():
        f.savefig(OUT / f"chart_{m}.png", dpi=160)

    setup = wrap(
        "Goal: measure how Temperature, Top-K and Top-P affect determinism, "
        "hallucination and structural correctness of locally run LLMs.\n\n"
        f"Models (Ollama): {', '.join(sorted(summary.model.unique()))}\n"
        f"Baseline: {BASELINE}\n"
        f"Sweeps (one parameter at a time): {SWEEPS}\n"
        f"Prompts: {len(PROMPTS)} (2 factual, 2 fake-entity, 2 JSON, 2 creative), "
        f"each run {REPS} times per setting.\n\n"
        "Metrics:\n"
        "  - Determinism: mean pairwise difflib similarity of the repeated runs.\n"
        "  - Hallucination rate: factual prompts fail if the expected keyword is "
        "missing; fake-entity prompts fail if the model does not refuse / admit "
        "it does not know.\n"
        "  - Strict hallucination rate: like the rate above, but a fake-entity "
        "answer counts as a refusal only if the \"I don't know\" is in the opening "
        "(first 300 characters) and no made-up details follow it.\n"
        "  - Structure: share of JSON prompts returning parseable JSON with all "
        "required keys.\n\n"
        "Hypotheses to test (not guaranteed outcomes): lower Temperature / Top-K / "
        "Top-P may produce more repeatable output and may affect refusal behaviour; "
        "higher Temperature may reduce determinism and may increase fabrication. "
        "JSON validity may or may not decline at higher Temperature. The discussion "
        "below reports what this experiment actually observed, including findings "
        "that were not confirmed. Results are limited by the small prompt set and "
        "three repetitions per setting.")
    obs = wrap("Automatically computed observations (temperature 0 vs 1.5):\n\n"
               + "\n\n".join(findings(summary)))
    disc = wrap(discussion(summary), 92)

    with PdfPages(PDF_PATH) as pdf:
        text_page(pdf, "Decoding Parameter & Sampling Analysis", setup, 9.5)
        for f in figs.values():
            pdf.savefig(f)
        for param in SWEEPS:
            table_page(pdf, summary, param)
        text_page(pdf, "Observations", obs, 10)
        text_page(pdf, "Discussion", disc, 9.5)
        script_pages(pdf)
    for f in figs.values():
        plt.close(f)
    print(f"Saved {PDF_PATH}, {SUMMARY_CSV} and charts in {OUT}/")


# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--models", nargs="+", default=MODELS)
    ap.add_argument("--reps", type=int, default=REPS)
    ap.add_argument("--report-only", action="store_true")
    args = ap.parse_args()

    globals()["REPS"] = args.reps
    raw = pd.read_csv(RAW_CSV) if args.report_only else run_benchmark(args.models, args.reps)
    summary = analyse(raw)
    build_report(summary)


if __name__ == "__main__":
    main()
```